HPC Programs

1.	Design and implement Parallel Breadth First Search and Depth First Search based on existing algorithms using OpenMP. Use a Tree or an undirected graph for BFS and DFS .

```
#include <iostream>
#include <vector>
#include <omp.h>

using namespace std;

class Graph {
    int V;
    vector<vector<int>> adj;

public:
    Graph(int V) {
        this->V = V;
        adj.resize(V);
    }

    void addEdge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    void parallelBFS(int start) {
        vector<bool> visited(V, false);
        vector<int> frontier;

        visited[start] = true;
        frontier.push_back(start);

        cout << "Parallel BFS from node " << start << ": ";

        double start_time = omp_get_wtime();

        while (!frontier.empty()) {
            for (int u : frontier)
                cout << u << " ";
```

```
        vector<int> next_frontier;

        #pragma omp parallel
        {
            vector<int> local_next;

            #pragma omp for nowait schedule(dynamic)
            for (int i = 0; i < (int)frontier.size(); i++) {
                for (int v : adj[frontier[i]]) {
                    bool should_visit = false;
                    #pragma omp critical
                    {
                        if (!visited[v]) {
                            visited[v] = true;
                            should_visit = true;
                        }
                    }
                    if (should_visit)
                        local_next.push_back(v);
                }
            }

            #pragma omp critical
            {
                next_frontier.insert(next_frontier.end(),
                                     local_next.begin(),
                                     local_next.end());
            }
        }

        frontier = next_frontier;
    }

    double end_time = omp_get_wtime();

    cout << "Time taken: " << (end_time - start_time) << " seconds\n";

    cout << endl;
}
```

```
void parallelDFS(int start) {
    vector<bool> visited(V, false);
    vector<int> stack;

    stack.push_back(start);

    cout << "Parallel DFS from node " << start << ": ";

    double start_time = omp_get_wtime();

    while (!stack.empty()) {
        int u = stack.back();
        stack.pop_back();

        if (visited[u]) continue;
        visited[u] = true;
        cout << u << " ";

        vector<int> to_push;

        #pragma omp parallel
        {
            vector<int> local_push;

            #pragma omp for nowait schedule(dynamic)
            for (int i = 0; i < (int)adj[u].size(); i++) {
                if (!visited[adj[u][i]])
                    local_push.push_back(adj[u][i]);
            }

            #pragma omp critical
            {
                to_push.insert(to_push.end(),
                               local_push.begin(),
                               local_push.end());
            }
        }

        for (int v : to_push)
```

```
                stack.push_back(v);
            }

            double end_time = omp_get_wtime();

            cout << "Time taken: " << (end_time - start_time) << " seconds\n";

            cout << endl;
        }
};

int main() {
    Graph g(6);

    g.addEdge(0, 1);
    g.addEdge(0, 2);
    g.addEdge(1, 3);
    g.addEdge(1, 4);
    g.addEdge(2, 5);

    g.parallelBFS(0);
    g.parallelDFS(0);

    return 0;
}
```

```
Parallel BFS from node 0: 0 1 2 5 3 4 Time taken: 0.00577397 seconds

Parallel DFS from node 0: 0 2 5 1 4 3 Time taken: 0.0132942 seconds
```

```
Parallel BFS from node 0: 0 1 2 5 3 4 Time taken: 0.000257462 seconds

Parallel DFS from node 0: 0 1 3 4 2 5 Time taken: 3.87409e-05 seconds
```

2.	Write a program to implement Parallel Bubble Sort and Merge sort using OpenMP. Use existing algorithms and measure the performance of sequential and parallel algorithms.

```
/*
 * EXECUTION INSTRUCTIONS (Debian-based distributions):
 *
```

```
 * i) Install g++ with OpenMP support:
 *   sudo apt update
 *   sudo apt install g++
 *
 * ii) Compile:
 *   g++ -fopenmp Code-2.cpp -o Code-2
 *
 * iii) Execute:
 *   ./Code-2
 * repository on KSKA Git:
https://git.kska.io/sppu-be-comp-content/HighPerformanceComputing
 **/

// BEGINNING OF CODE
#include <iostream>
#include <vector>
#include <cstdlib>
#include <omp.h>
#include <algorithm>

using namespace std;

void printArray(const vector<int>& arr) {
    for (int num : arr)
        cout << num << " ";
    cout << endl;
}

void printArray10(const vector<int>& arr) {
    for (int i = 0; i < min(10, (int)arr.size()); i++)
        cout << arr[i] << " ";
    cout << "... (truncated)" << endl;
}

// Bubble Sort

// Sequential bubble sort.
// Sorts the array using bubble sort by repeatedly swapping adjacent
elements.
void sequentialBubbleSort(vector<int>& arr) {
```

```
    int n = arr.size();
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1])
                swap(arr[j], arr[j + 1]);
        }
    }
}

// Parallel bubble sort using odd-even transposition.
// Standard bubble sort cannot be parallelized directly: thread on index j
// and thread on index j+1 would both touch arr[j+1] simultaneously (data
race).
// Odd-even transposition alternates between two phases each pass:
//   Phase 0 (even): compare pairs (0,1), (2,3), (4,5), ...
//   Phase 1 (odd):  compare pairs (1,2), (3,4), (5,6), ...
// Within each phase every pair is independent, so threads never share
elements.
void parallelBubbleSort(vector<int>& arr) {
    int n = arr.size();
    for (int i = 0; i < n; i++) {
        // i % 2 selects even phase (0) or odd phase (1).
        // The starting index of the first pair in each phase matches i%2.
        #pragma omp parallel for
        for (int j = i % 2; j < n - 1; j += 2) {
            if (arr[j] > arr[j + 1])
                swap(arr[j], arr[j + 1]);
        }
    }
}

// Merge Sort

// Merges two sorted halves arr[left..mid] and arr[mid+1..right] in place.
void merge(vector<int>& arr, int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    vector<int> L(n1), R(n2);
    for (int i = 0; i < n1; i++) L[i] = arr[left + i];
```

```
    for (int i = 0; i < n2; i++) R[i] = arr[mid + 1 + i];

    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2)
        arr[k++] = (L[i] <= R[j]) ? L[i++] : R[j++];

    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];
}

void sequentialMergeSort(vector<int>& arr, int left, int right) {
    if (left >= right) return;
    int mid = left + (right - left) / 2;
    sequentialMergeSort(arr, left, mid);
    sequentialMergeSort(arr, mid + 1, right);
    merge(arr, left, mid, right);
}

// Parallel merge sort using OpenMP tasks.
// "#pragma omp parallel sections" inside a recursive function would spawn
a
// new thread team at every level of recursion, hundreds of thousands of
teams
// for a large array, causing enormous overhead and likely a crash.
// Tasks are lighter: the runtime schedules them across an existing thread
pool.
// The depth cutoff switches to sequential below a threshold to avoid
spawning
// tasks so small that the overhead exceeds the work itself.
void parallelMergeSortHelper(vector<int>& arr, int left, int right) {
    if (left >= right) return;

    int mid = left + (right - left) / 2;

    // Use threshold instead of depth (more reliable)
    if (right - left < 1000) {
        sequentialMergeSort(arr, left, right);
        return;
    }
```

```
    #pragma omp task shared(arr)
    parallelMergeSortHelper(arr, left, mid);

    #pragma omp task shared(arr)
    parallelMergeSortHelper(arr, mid + 1, right);

    #pragma omp taskwait   // ensure both halves are sorted

    merge(arr, left, mid, right);
}

void parallelMergeSort(vector<int>& arr, int left, int right) {
    #pragma omp parallel
    {
        #pragma omp single
        parallelMergeSortHelper(arr, left, right);
    }
}

// Main function

int main() {
    int n = 10000; // Adjust this to specify the number of elements.
    vector<int> arr(n);

    for (int i = 0; i < n; i++)
        arr[i] = rand() % 10000;

    double start, end;
    double time_seq_bubble, time_par_bubble;
    double time_seq_merge, time_par_merge;

    // --- Sequential Bubble Sort ---
    vector<int> seqArr = arr;
    start = omp_get_wtime();
    sequentialBubbleSort(seqArr);
    end = omp_get_wtime();
    time_seq_bubble = end - start;
    cout << "Sequential Bubble Sort time: " << time_seq_bubble << "
seconds" << endl;
```

```
    // --- Parallel Bubble Sort ---
    vector<int> parArr = arr;
    start = omp_get_wtime();
    omp_set_num_threads(2);
    parallelBubbleSort(parArr);
    end = omp_get_wtime();
    time_par_bubble = end - start;
    cout << "Parallel Bubble Sort time: " << time_par_bubble << " seconds"
<< endl;

    cout << "Bubble Sort Speedup (Sequential / Parallel) = " <<
(time_seq_bubble / time_par_bubble) << "x" << endl;

    cout << "\nParallel Bubble Sort (first 10 elements): ";
    printArray10(parArr);

    if (is_sorted(parArr.begin(), parArr.end()))
        cout << "Array is sorted correctly\n";

    cout << "Parallel Bubble Sort time: " << time_par_bubble << " seconds"
<< endl;


    // --- Sequential Merge Sort ---
    seqArr = arr;
    start = omp_get_wtime();
    sequentialMergeSort(seqArr, 0, n - 1);
    end = omp_get_wtime();
    time_seq_merge = end - start;
    cout << "\nSequential Merge Sort time: " << time_seq_merge << "
seconds" << endl;

    // --- Parallel Merge Sort ---
    parArr = arr;
    start = omp_get_wtime();
    omp_set_num_threads(8);
    parallelMergeSort(parArr, 0, n - 1);
    end = omp_get_wtime();
    time_par_merge = end - start;
```

```
    cout << "Parallel Merge Sort time: " << time_par_merge << " seconds"
<< endl;

    cout << "Merge Sort Speedup (Sequential / Parallel) = " <<
(time_seq_merge / time_par_merge) << "x" << endl;


    cout << "\nParallel Merge Sort (first 10 elements): ";
    printArray10(parArr);

    if (is_sorted(parArr.begin(), parArr.end()))
        cout << "Array is sorted correctly\n";

    cout << "Parallel Merge Sort time: " << time_par_merge << " seconds"
<< endl;

    return 0;
}
```

```
Sequential Bubble Sort time: 0.890277 seconds
Parallel Bubble Sort time: 0.687411 seconds
Bubble Sort Speedup (Sequential / Parallel) = 1.29512x

Parallel Bubble Sort (first 10 elements): 0 1 1 1 3 4 5 5 5 6 ... (truncated)
Array is sorted correctly
Parallel Bubble Sort time: 0.687411 seconds

Sequential Merge Sort time: 0.0059242 seconds
Parallel Merge Sort time: 0.00261949 seconds
Merge Sort Speedup (Sequential / Parallel) = 2.26158x

Parallel Merge Sort (first 10 elements): 0 1 1 1 3 4 5 5 5 6 ... (truncated)
Array is sorted correctly
Parallel Merge Sort time: 0.00261949 seconds
```

3.	Implement Min, Max, Sum and Average operations using Parallel Reduction.

```
#include <iostream>
#include <vector>
#include <omp.h>
#include <cstdlib>
```

```
using namespace std;

int main(){

    int n = 1000000;
    vector<int> nums(n);
    for(int i=0;i<n;i++){
        nums[i] = rand() % 10000;
    }
    cout<<"Input: "<<n<<endl;

    long long sums,sump;
    int mins,maxs,minp,maxp;
    double avgs,avgp;
    double start,end,times,timep;

    mins, maxs = nums[0];
    sums = 0;
    start = omp_get_wtime();
    for(int i=0;i<n;i++){
        if(nums[i]<mins)
            mins = nums[i];
        if(nums[i]>maxs)
            maxs = nums[i];
        sums+=nums[i];
    }
    end = omp_get_wtime();
    avgs = (double) sums/n;
    times = end-start;
    cout<<"Time sequential: "<<times<<endl;

    minp, maxp = nums[0];
    sump = 0;
    start = omp_get_wtime();
    #pragma omp parallel for reduction(min: minp) reduction(max: maxp)
reduction(+:sump)
    for(int i=0;i<n;i++){
        if(nums[i]<minp)
            minp = nums[i];
        if(nums[i]>maxp)
```

```
            maxp = nums[i];
        sump+=nums[i];
    }
    end = omp_get_wtime();
    avgp = (double) sump/n;
    timep = end-start;
    cout<<"Time parallel: "<<timep<<endl;

    // --- Output ---
    cout << "--- Sequential Computation ---" << endl;
    cout << "Minimum  : " << mins << endl;
    cout << "Maximum  : " << maxs << endl;
    cout << "Sum      : " << sums << endl;
    cout << "Average  : " << avgs << endl;
    cout << "Time     : " << times << " seconds" << endl;

    cout << "\n--- Parallel Computation ---" << endl;
    cout << "Minimum  : " << minp << endl;
    cout << "Maximum  : " << maxp << endl;
    cout << "Sum      : " << sump << endl;
    cout << "Average  : " << avgp << endl;
    cout << "Time     : " << timep << " seconds" << endl;

    cout << "\nSpeedup (Sequential / Parallel) = " << (times / timep) <<
"x" << endl;

    return 0;
}
```

```
Input: 1000000
Time sequential: 0.0129452
Time parallel: 0.00716469
--- Sequential Computation ---
Minimum   : 0
Maximum   : 9999
Sum       : 5000491283
Average   : 5000.49
Time      : 0.0129452 seconds

--- Parallel Computation ---
Minimum   : 0
Maximum   : 9999
Sum       : 5000491283
Average   : 5000.49
Time      : 0.00716469 seconds

Speedup (Sequential / Parallel) = 1.80681x


...Program finished with exit code 0
Press ENTER to exit console.
```

==========

Functions

```cpp
#include <iostream>
#include <vector>
#include <omp.h>
#include <cstdlib>

using namespace std;

// Function to generate random numbers
vector<int> generateData(int n) {
    vector<int> nums(n);
    for (int i = 0; i < n; i++)
        nums[i] = rand() % 10000;
    return nums;
```

```
}

// Sequential computation
void sequentialCompute(const vector<int>& nums, int &min_val, int
&max_val, long long &sum, double &avg, double &time_taken) {
    int n = nums.size();
    min_val = max_val = nums[0];
    sum = 0;

    double start = omp_get_wtime();

    for (int i = 0; i < n; i++) {
        if (nums[i] < min_val) min_val = nums[i];
        if (nums[i] > max_val) max_val = nums[i];
        sum += nums[i];
    }

    double end = omp_get_wtime();
    time_taken = end - start;

    avg = (double)sum / n;
}

// Parallel computation
void parallelCompute(const vector<int>& nums, int &min_val, int &max_val,
long long &sum, double &avg, double &time_taken) {
    int n = nums.size();
    min_val = max_val = nums[0];
    sum = 0;

    double start = omp_get_wtime();

    #pragma omp parallel for reduction(min: min_val) reduction(max:
max_val) reduction(+: sum)
    for (int i = 0; i < n; i++) {
        if (nums[i] < min_val) min_val = nums[i];
        if (nums[i] > max_val) max_val = nums[i];
        sum += nums[i];
    }
```

```
    double end = omp_get_wtime();
    time_taken = end - start;

    avg = (double)sum / n;
}

// Function to print results
void printResults(string title, int min_val, int max_val, long long sum,
double avg, double time_taken) {
    cout << "--- " << title << " ---" << endl;
    cout << "Minimum  : " << min_val << endl;
    cout << "Maximum  : " << max_val << endl;
    cout << "Sum      : " << sum << endl;
    cout << "Average  : " << avg << endl;
    cout << "Time     : " << time_taken << " seconds" << endl;
}

int main() {
    int n = 1000000;

    // Generate data
    vector<int> nums = generateData(n);

    cout << "Input: " << n << " random integers in the range [0, 9999].\n"
<< endl;

    // Variables
    int min_seq, max_seq, min_par, max_par;
    long long sum_seq, sum_par;
    double avg_seq, avg_par;
    double time_seq, time_par;

    // Sequential
    sequentialCompute(nums, min_seq, max_seq, sum_seq, avg_seq, time_seq);

    // Parallel
    parallelCompute(nums, min_par, max_par, sum_par, avg_par, time_par);

    // Output
```

```
    printResults("Sequential Computation", min_seq, max_seq, sum_seq,
avg_seq, time_seq);
    cout << endl;
    printResults("Parallel Computation", min_par, max_par, sum_par,
avg_par, time_par);

    cout << "\nSpeedup (Sequential / Parallel) = " << (time_seq /
time_par) << "x" << endl;

    return 0;
}
```

4.	Write a CUDA Program for : 1. Addition of two large vectors 2. Matrix Multiplication using CUDA C

Practical-4 (Vector Addition and Matrix Multiplication)

Problem Statement: Write a CUDA Program for:

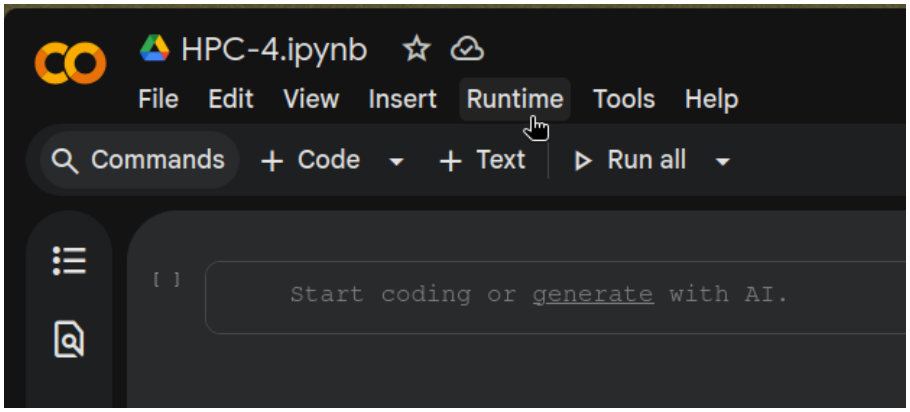
1. Addition of two large vectors
2. Matrix Multiplication using CUDA C

Pre-requisities

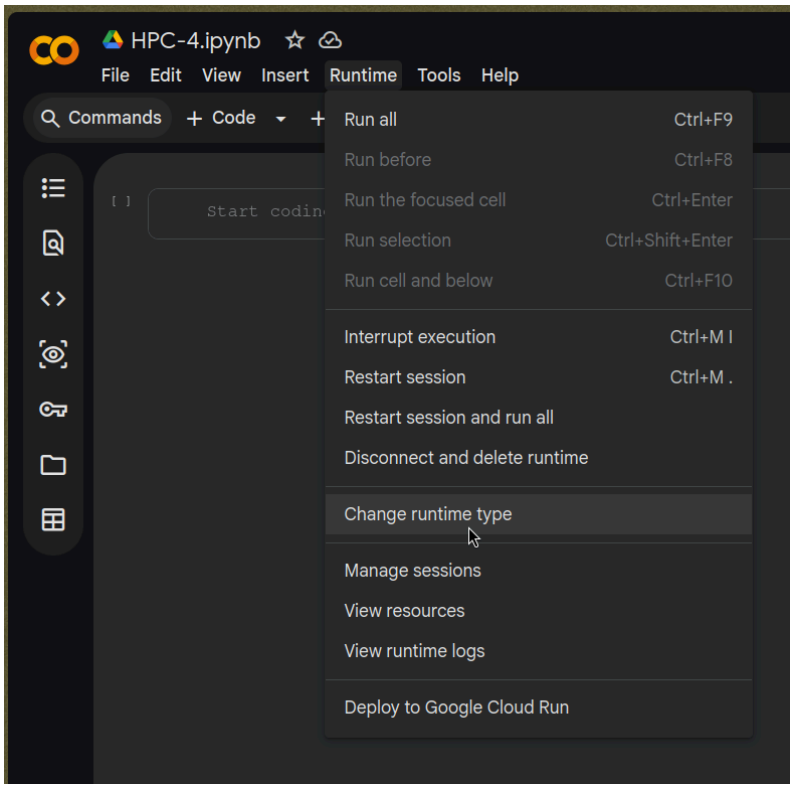
1. Open Google Colab
2. Create a new Jupyter Notebook

Steps

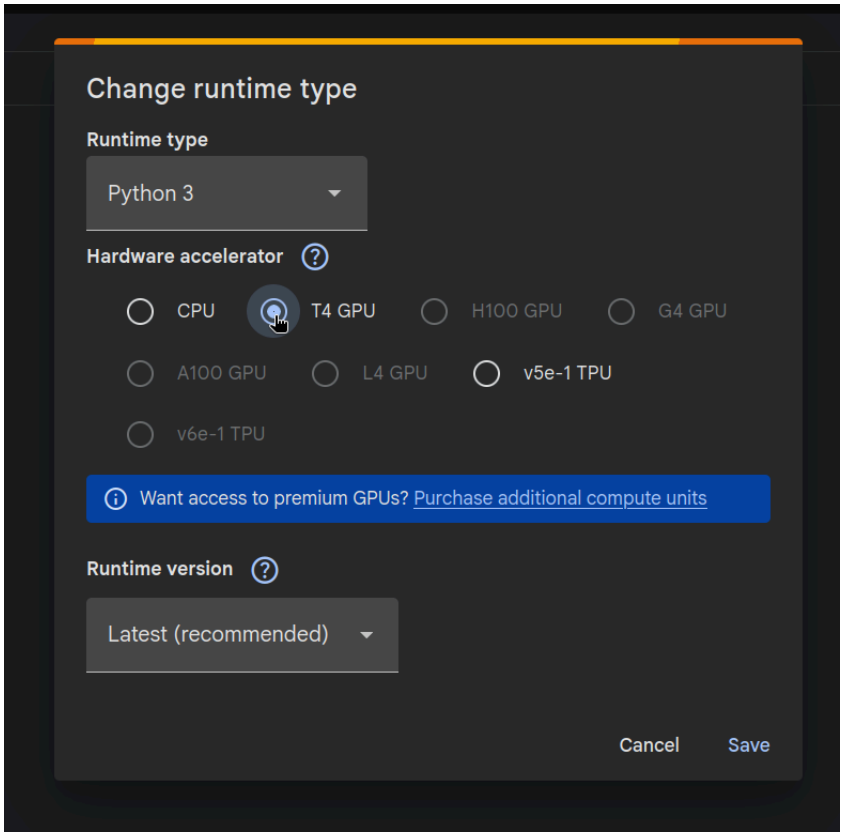
1. After creating a new Jupyter notebook, click on "Runtime" in the navbar:



2. Then, choose "Change runtime type":



3. Select "T4 GPU", and save:



4. Check if `nvcc` is installed:

```
!nvcc --version
```

5. Install `nvcc4jupyter`:

```
!pip install nvcc4jupyter
# Or if the above command fails, comment the above
line and run
# !pip install
git+https://git.kska.io/notkshitij/nvcc.git
```

6. Load it:

```
%load_ext nvcc4jupyter
```

7. Paste the below code in a new code block:

```
%%writefile cuda_program.cu
#include <iostream>
#include <cuda.h>

using namespace std;

#define BLOCK_SIZE 2

// Vector Addition Kernel
// Each thread computes a single element of C = A + B.
__global__ void vectorAdd(int *A, int *B, int *C, int N) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    // Guard against threads beyond the vector size (when N is
not a multiple
    // of the block size, some threads in the last block are out
of range).
    if (i < N)
        C[i] = A[i] + B[i];
}

// Matrix Multiplication Kernel
// Each thread computes a single element of C = A * B.
// Thread (row, col) sums the dot product of row `row` of A with
column `col` of B.
__global__ void matrixMul(float *A, float *B, float *C, int N) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;

    float sum = 0.0f;
    for (int n = 0; n < N; ++n)
        sum += A[row * N + n] * B[n * N + col];

    C[row * N + col] = sum;
}

// Vector Addition
```

```
void runVectorAddition() {
    int N;
    cout << "\n=== Vector Addition ===" << endl;
    cout << "Enter vector size: ";
    cin >> N;

    int size = N * sizeof(int);

    // Host allocation and initialisation
    int *hA = new int[N];
    int *hB = new int[N];
    int *hC = new int[N];

    for (int i = 0; i < N; i++) {
        hA[i] = i;
        hB[i] = i * 2;
    }

    cout << "\nVector A: ";
    for (int i = 0; i < N; i++) cout << hA[i] << " ";
    cout << "\nVector B: ";
    for (int i = 0; i < N; i++) cout << hB[i] << " ";
    cout << endl;

    // Device allocation and transfer
    int *dA, *dB, *dC;
    cudaMalloc(&dA, size);
    cudaMalloc(&dB, size);
    cudaMalloc(&dC, size);

    cudaMemcpy(dA, hA, size, cudaMemcpyHostToDevice);
    cudaMemcpy(dB, hB, size, cudaMemcpyHostToDevice);

    // Launch with enough blocks to cover all N elements.
    // (N + BLOCK_SIZE - 1) / BLOCK_SIZE rounds up so we don't
miss the tail.
    int numBlocks = (N + BLOCK_SIZE - 1) / BLOCK_SIZE;
    vectorAdd<<<numBlocks, BLOCK_SIZE>>>(dA, dB, dC, N);

    cudaMemcpy(hC, dC, size, cudaMemcpyDeviceToHost);
```

```
    cout << "Result A + B: ";
    for (int i = 0; i < N; i++) cout << hC[i] << " ";
    cout << endl;

    delete[] hA;
    delete[] hB;
    delete[] hC;
    cudaFree(dA);
    cudaFree(dB);
    cudaFree(dC);
}

// Matrix Multiplication
void runMatrixMultiplication() {
    int K, N;
    cout << "\n=== Matrix Multiplication ===" << endl;
    cout << "Enter K (matrix will be N x N where N = K * " <<
BLOCK_SIZE << "): ";
    cin >> K;
    N = K * BLOCK_SIZE;

    cout << "Matrix size: " << N << " x " << N << endl;
    int size = N * N * sizeof(float);

    // Host allocation and initialisation
    float *hA = new float[N * N];
    float *hB = new float[N * N];
    float *hC = new float[N * N];

    for (int j = 0; j < N; j++) {
        for (int i = 0; i < N; i++) {
            hA[j * N + i] = 2;
            hB[j * N + i] = 4;
        }
    }

    cout << "\nMatrix A:\n";
    for (int row = 0; row < N; row++) {
        for (int col = 0; col < N; col++)
            cout << hA[row * N + col] << " ";
        cout << endl;
```

```
    }

    cout << "\nMatrix B:\n";
    for (int row = 0; row < N; row++) {
        for (int col = 0; col < N; col++)
            cout << hB[row * N + col] << " ";
        cout << endl;
    }

    // Device allocation and transfer
    float *dA, *dB, *dC;
    cudaMalloc(&dA, size);
    cudaMalloc(&dB, size);
    cudaMalloc(&dC, size);

    cudaMemcpy(dA, hA, size, cudaMemcpyHostToDevice);
    cudaMemcpy(dB, hB, size, cudaMemcpyHostToDevice);

    // threadBlock: BLOCK_SIZE x BLOCK_SIZE threads per block.
    // grid: K x K blocks, so total threads = N x N (one per
output element).
    dim3 threadBlock(BLOCK_SIZE, BLOCK_SIZE);
    dim3 grid(K, K);
    matrixMul<<<grid, threadBlock>>>(dA, dB, dC, N);

    cudaMemcpy(hC, dC, size, cudaMemcpyDeviceToHost);

    cout << "\nResult C = A * B:\n";
    for (int row = 0; row < N; row++) {
        for (int col = 0; col < N; col++)
            cout << hC[row * N + col] << " ";
        cout << endl;
    }

    delete[] hA;
    delete[] hB;
    delete[] hC;
    cudaFree(dA);
    cudaFree(dB);
    cudaFree(dC);
}
```

```
int main() {
    runVectorAddition();
    runMatrixMultiplication();

    cout << "\nFinished." << endl;
    return 0;
}
```

8. Compile and run:

```
!nvcc cuda_program.cu -o cuda_program && ./cuda_program
```

Sample output

```
=== Vector Addition ===
Enter vector size: 2

Vector A: 0 1
Vector B: 0 2
Result A + B: 0 3

=== Matrix Multiplication ===
Enter K (matrix will be N x N where N = K * 2): 3
Matrix size: 6 x 6

Matrix A:
2 2 2 2 2 2
2 2 2 2 2 2
2 2 2 2 2 2
2 2 2 2 2 2
2 2 2 2 2 2
2 2 2 2 2 2

Matrix B:
4 4 4 4 4 4
```

```
4 4 4 4 4 4
4 4 4 4 4 4
4 4 4 4 4 4
4 4 4 4 4 4
4 4 4 4 4 4

Result C = A * B:
48 48 48 48 48 48
48 48 48 48 48 48
48 48 48 48 48 48
48 48 48 48 48 48
48 48 48 48 48 48
48 48 48 48 48 48

Finished.
```